Lecture 12 - More pseudorandomness and Hardness vs Randomness

Lecture 12 - More pseudorandomness and Hardness vs Randomness
- Pseudorandom functions
- Derandomization
- Derandomization using PRGs
  - Average-case Hardness
  - Baby steps towards Nisan & Wigderson

So, we have had a bit of a ride so far. What did we learn?

- We learned what a computer is, as a mathematical object.
- We learned how to measure different computational resources - time and space - under different notions of computation - deterministic, non-deterministic, randomized.
- We understood that we can classify computational problems by how many resources are needed to solve them. This gives rise to complexity classes, like P, BPP, NP, PSPACE, L, NL, etc
- We learned that sometimes, a single problem is a perfect representative of the computational complexity of all problems in a given complexity class. E.g. SAT for NP, reachability for NL, TQBF for PSPACE.
- We learned about reductions, how can one make rigorous the statement "Problem A is easier than problem B".
- We saw hierarchy theorems - more resources allow us to solve more problems.
- Yet, mysteriously, we cannot prove, e.g., that 3SUM is not computable in linear time.
- We saw the relativization barrier, that seems to imply that the diagonalization techniques that allow us to prove hierarchy theorems will not be enough to prove that P != NP.
- We started looking at a more combinatorial version of computation: Boolean formulas and circuits.
- We saw a few examples of combinatorial lower bounds, eventually we constructed a function f that cannot be computed by sub-cubic size ($n^{3-\varepsilon}$) formulas.
- Then we took what appeared to be a hard turn: we looked at cryptography. We learned about private-key cryptography, one-way functions, pseudorandom generators, and the relation between them.

For the rest of the course:

- Today we will go deeper into pseudorandom generators. We will talk about pseudorandom functions, and pseudorandom generators of exponential stretch (!), which will allow us to show that $\mathsf{P} = \mathsf{BPP}$ under reasonable assumptions.
- Next week we will finish the derandomization part, and start discussing the natural proofs barrier, which we should finish on the last week, or if we have time, then on the last week we can do something else.

Pseudorandom functions

A pseudorandom function is a generaliztion of a pseudorandom generator:

- In a pseudorandom generator, the distinguisher gets a string y of size $\mathsf{poly}(n)$, and wishes to estimate whether y is a truly random string, versus $y = G(x)$ for a random x.
- In a pseudorandom function setting, the distinguisher has oracle access to a function $F : \{0,1\}^n \to \{0,1\}^n$. The distinguisher can make queries to the function, as many as it wants to within the time he has available. The distinguisher's task is to decide whether F is truly a random function (meaning each output $F(x)$ is chosen independently and uniformly), versus when F is a "pseudorandom function".

Definition. A *pseudorandom function* if a (family of) function(s) $F : \{0,1\}^m \times \{0,1\}^n \to \{0,1\}^n$, such that $m \leq \mathsf{poly}(n)$, and $F(s,x) = F_s(x)$ can be computed in time polynomial in n, and for every probabilistic polynomial-time algorithm A with an oracle,

$$|\Pr_s[A^{F_s}(0^n) = 1] - \Pr_R[A^R(0^n) = 1]| = n^{-\omega(1)} \text{ (i.e., is negligible)}$$

where R is a uniformly chosen random function in $\{0,1\}^n \to \{0,1\}^n$.

Theorem (Goldreich, Goldwasser, and Micali). Suppose there exists a secure PRG $G : \{0,1\}^n \to \{0,1\}^{2n}$. Let $G(z) = G_0(z)G_1(z)$ be the two halves of the output of G, so each is n-bits long.

Then the following is a secure pseudorandom function $F : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n$:

$$F_s(x) = G_{x_n} \circ \cdots \circ G_{x_1}(s).$$

Proof.

Let us argue that the above family F_s is a pseudorandom function family.

We do this in the usual way: suppose by hypothesis that we have a distinguisher that breaks the pseudorandom function family F, and try to use it to construct a distinguisher that breaks the pseudorandom generator G.

So, suppose that we had an algorithm A which distinguishes our pseudorandom F from a truly random R, i.e.,

$$|\Pr[A^{F_s}(1^n) = 1] - \Pr[A^R(1^n) = 1]| > \frac{1}{n^c}$$

for some constant c.

Let us think of what the algorithm is doing when it queries F_s.

F_s itself is computable by an algorithm B that makes n calls to G.

```
B(s,x):
    (y0, y1) = G(S)
    (y0,y1) = G(y0) if x1=0 else G(y1)
    (y0,y1) = G(y0) if x2=0 else G(y1)
    ...
    (y0,y1) = G(y0) if x{n-1}=0 else G(y1)
    output y0 if xn=0 else y1
```

So, in total, one can think of A as making at most nT calls to G, where T is the running time of A.

We then create hybrids O_i, where the last i calls to G are replaced with truly random $2n$-bit strings.

A random function is now equivalent to O_{nT}, and the pseudorandom function is equivalent to O_0.

It then follows in the usual hybrid argument way, that if A distinguishes O_0 from O_{nT}:

$$|\Pr[A^{O_0}(1^n) = 1] - \Pr[A^{O_{nT}}(1^n) = 1]| > \frac{1}{n^c}$$

then there must exist an i such that A distinguishes O_i from O_{i+1}:

$$|\Pr[A^{O_i}(1^n) = 1] - \Pr[A^{O_{i+1}}(1^n) = 1]| > \frac{1}{n^c \cdot nT}$$

This is still a non-negligible gap since T is polynomial in n. Now notice that, the only difference between O_i and O_{i+1} is on whether a certain $2n$ bit string is truly random versus being the output of a call to G on a truly random string. This, then, gives us a distinguisher for fooling G itself:

- On input y which we wish to know whether it is truly random versus equal to $G(x)$ for truly random x, we run $A(1^n)$, so that the last i calls to G are replaced with truly random strings, the $i+1$-th (counting from the end) call to G is replaced with y, and the remaining ones are evaluated using G.
- By the above, this distinguisher will distinguish the two cases for y with non-negligible gap.

Now for derandomization.

Derandomization

ok, now we would like to show how to derandomize a randomized algorithm.

I.e., we would like to show, perhaps under some reasonable assumption, that $\mathsf{P} = \mathsf{BPP}$.

I find the results we are about to see very beautiful.

Because there is this intuitive idea that:

- Something random is something which is hard to predict.
- A function which is hard to compute can be thought of as a map $x \mapsto f(x)$ such that $f(x)$ is *hard to predict* when given x.

The magic of the hardness vs randomness result that we will see in this an the next class is that this intuition can be made into a theorem!

We will show, for example, that if there exist a function $f \in \mathsf{DTIME}(2^{100n})$ which is "hard to compute" by circuits of size $2^{\frac{n}{100}}$, say, then $\mathsf{P} = \mathsf{BPP}$.

Definition 9.8 (PRG, cryptographic setting). Let $G : \{0,1\}^m \to \{0,1\}^{l(m)}$ be a $poly(m)$-time computable function (for all m) where $l(m) > m$. We say G is a secure pseudrandom generator of stretch $l(m)$ if for every probabilistic polynomial-time algorithm A,

$$|\Pr[A(G(U_m))] - \Pr[A(U_{\ell(m)})]| = n^{-\omega(1)}.$$

Definition 20.2 (PRG, derandomization setting). Let $G : \{0,1\}^m \to \{0,1\}^\ell$ be a 2^m-time computable function (more precisely, we have one function for each m, and $\ell = \ell(m)$ is a function of m). We say G is an (S, ε)-secure PRG (S and ε are also functions of m) if for every circuit C of size at most S,

$$|\Pr[C(G(U_m)) = 1] - \Pr[C(U_\ell)) = 1]| \leq \varepsilon$$

So what changed?three things:

1. We allow G to run in time exponential in its input. This will make sense particularly when $l(m)$ is exponential in m!
2. The generator G should fool non-uniform distinguishers, i.e., circuits.
3. We precisely keeping track of the size S of the distinguisher circuits, and the fooling parameter ε, instead of just saying "polynomial" and "non-negligible".

Otherwise the idea is the same.

Derandomization using PRGs

Thm. If there exists a family $G : \{0,1\}^m \to \{0,1\}^\ell$ of $(\ell^3, 1/10)$-secure PRGs, then $\mathsf{BPTIME}(\ell) \subseteq \mathsf{DTIME}(2^{O(m)}\mathsf{poly}(\ell))$.

Proof.

Take any probabilistic algorithm $A(x, r)$ --- so x is the input and r is the randomness --- which runs in time ℓ. As we saw, we can construct a circuit C which simulates $A(x, r)$, and the size $|C|$ is $O(\ell^2)$.

But we can do more. Since we have the circuit C, then for any input x that we want, we can hardwire it into the circuit, to obtain another circuit C' such that $C'(r) = C(x, r)$. The size of the circuit didn't grow.

If we assume that G is secure against circuits of this size, then

$$|\Pr[C'(U_\ell) = 1] - \Pr[C'(G(U_m)) = 1]| \leq 1/10.$$

But because C' simulates $A(x, \cdot)$, this is equivalent to saying that $A(x, G(U_m))$ and $A(x, U_\ell)$ have roughly the same acceptance probability. Hence, an algorithm for knowing whether $A(x, r)$ accepts or rejects: we just go over every single seed $s \in \{0,1\}^m$, compute $r = G(s)$, and count how many times $A(x, G(s))$ accepts. If the original probabilistic algorithm accepts x, then $A(x, r) = 1$ with probability at least 2/3. Hence $A(x, G(s)) = 1$ with probability greater than $1/2$. Otherwise, $A(x, G(s)) = 1$ with probability less than $1/2$. So this allows to tell whether A accepts x or not.

This deterministic algorithm takes time $2^{O(m)}\mathsf{poly}(\ell)$. □

Corollary. If for some $\delta > 0$, there exists a family $G : \{0,1\}^m \to \{0,1\}^\ell$ of $(\ell^3, 1/10)$-secure PRGs, where $\ell = 2^{\delta m}$, then $\mathsf{P} = \mathsf{BPP}$.

Proof. As above. A polynomial-time algorithm only uses $\ell = \mathsf{poly}(n)$ bits, and in this case we can let $m = O(\log n)$. Going over all seeds now only takes polynomial time!

Average-case Hardness

Let $f : \{0,1\}^n \to \{0,1\}$. We define the *average-case hardness* of f, denoted $\mathsf{H}_{\text{avg}}(f)$, to be the largest number S such that every circuit of size $\leq S$ has

$$\Pr_x[C(x) = f(x)] \leq \frac{1}{2} + \frac{1}{S}.$$

By comparison, $\mathsf{S}(f)$, the smallest size of a circuit for computing f, is equal to 1 plus the largest number S such that every circuit of size $\leq S$ has

$$\Pr_x[C(x) = f(x)] < 1.$$

In general, it is easier to compute f better than random guessing, than it is to compute f correctly on every input. Hence the statement $\mathsf{H}_{\text{avg}}(f) \geq S$ is a stronger statement, it means that f is *harder*, than the statement $\mathsf{S}(f) \geq S$.

Theorem (Nisan & Wigderson, 1988). If there exists $\varepsilon > 0$ and $f \in E = \mathsf{DTIME}(2^{O(n)})$ such that $\mathsf{H}_{\text{avg}}(f) \geq 2^{\varepsilon n}$, then $\mathsf{P} = \mathsf{BPP}$.

Theorem (Impagliazzo & Wigderson). If there exists $\varepsilon > 0$ and $f \in E$ with $\mathsf{S}(f) \geq 2^{\varepsilon n}$, then $\mathsf{P} = \mathsf{BPP}$.

Baby steps towards Nisan & Wigderson

Proposition. Suppose that $\mathsf{H}_{\text{avg}}(f) \geq n^4$. Then the function $G : \{0,1\}^n \to \{0,1\}^{n+1}$, given by $G(z) = zf(z)$, is an $(\frac{n^4}{3}, \frac{1}{n^3})$-secure PRG.

First we need to observe that Yao's "unpredictability implies pseudorandomness" theorem also works for circuits (HW). Namely, we have the following:

Yao's unpredictability implies pseudorandomness for circuits. If, for ever $i \in [m]$, every circuit P of size $|P| \leq 3S$ fails to predict the i-th bit of G when given the previous bits, i.e.:

$$\Pr_x[P(G(x)_1 \dots G(x)_{i-1}) = G(x)_i] \leq \frac{1}{2} + \frac{\varepsilon}{n}$$

then G is (S, ε)-secure.

Again, this is proven by the converse. If we had an ε-distinguisher for G of size S, then (using a hybrid argument) we could get an ε/n predictor for G for one of the bits of G of size less than $3S$ (HW).

So to prove that the above $G(z) = zf(z)$ is a PRG, we need only prove that its output bits are unpredictable. The first m bits are clearly unpredictable as they are truly random. What would predicting the last bit mean? It would mean that there exists a circuit P with $\Pr_z[P(z) = f(z)] \geq \frac{1}{2} + \delta$. If we assume that $\mathsf{H}_{\text{avg}}(f) \geq n^4$, say, we must have either $|P| \geq n^4$ or $\delta < \frac{1}{n^4}$. It then follows from Yao's theorem that G is $(n^4/3, 1/n^3)$-secure.

How to get more bits than just 1 extra bit?

$G(x_1, x_2) = x_1 f(x_1) x_2 f(x_2)$

This stretches $2n$ bits into $2n + 2$ bits.

Clearly the x_i cannot be predicted as they are truly random. I cannot predict $f(x_1)$ from x_1 by the argument we just did.

How about predicting $f(x_2)$, when given x_1, $f(x_1)$ and x_2?

Ok, let's see if we can prove that this doesn't help. Suppose we had a predictor P for this task. Let's break down its probability of success:

$$\frac{1}{2} + \varepsilon \leq \Pr[P(x_1, f(x_1), x_2) = f(x_2)] = \underset{x_1}{E}[\Pr_{x_2}[P(x_1, f(x_1), x_2) = f(x_2) \mid x_1]]$$

Then there must exist some fixed choice of x_1 such that the probability inside the expectation is at least the expectation. But then, we can just take the predictor $P'(x_2) = P(x_1, f(x_1), x_2)$ (for x_1 equal to this particular fixed choice). And then we have $\Pr_{x_2}[P'(x_2) = f(x_2)] \geq \frac{1}{2} + \varepsilon$. And the size of P' didn't increase at all!

Outra ideia: $G(x) = xf(x)f(x_2 \dots x_n f(x))$

How about the following:

$G(x_0, x_1, x_2) = x_0 x_1 x_2 f(x_0 x_1) f(x_0 x_2)$

$|x_1| = n - \log n = |x_2|$, $|x_0| = \log n$

Let's try to show that this is still hard to predict.

The first part $x_0 x_1 x_2$ is truly random hence unpredictable.

The first application $f(x_0 x_1)$ is unpredictable if f is hard on average, as we just saw.

How about the second application.

Suppose we had a predictor P:

$$\frac{1}{2} + \varepsilon \leq \Pr[P(x_0, x_1, x_2, f(x_0, x_1)) = f(x_0, x_2)]$$

By the same argument as above, there exists a fixed choice of x_1 that attains the same prediction probability. The crucial idea: $f(x_0, x_1)$ is now a function of x_0 only, since we have fixed x_1 to some fixed value. And $|x_0| = \log n$, hence the function $x_0 \mapsto f(x_0, x_1)$ can be computed by a circuit C of size $O(n)$!

Then we can take the predictor:

$$P'(x_0, x_2) = P(x_0, x_1, x_2, C(x_0))$$

(here x_1 is our fixed choice) and this will have size $|P| + O(n)$ only. And still predicts $f(x_0 x_2)$ with the same success probaiblity.